

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

Источники данных. Чтение и запись текстовых файлов: классы StreamReader и
StreamWriter.

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г. В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ.....	3
2 ЗАДАНИЕ	4
3 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ	5
4 ИСХОДНЫЙ КОД ПРОГРАММЫ	7
5 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ	26
6 ВЫВОД.....	28

1 Цель работы

Изучить способы читать и записывать текстовые файлы с помощью классов и их перегруженных версий из пространства Stream.IO.

2 Задание

Выполните все задания, сформулированные в тексте лабораторной работы для собственных классов. Разработайте и протестируйте перегруженные версии методов для чтения/записи данных, вычислительные методы, например, вычисление возраста, стажа, срока годности....

На примере разработанных методов исследуйте механизмы передачи параметров в методы: по значению, по ссылке, выходные параметры, необязательные параметры.

Для тестирования методов обязательно разрабатывайте интерфейс пользователя с использованием элементов формы, оптимальных отображаемым типам данных. Старайтесь программировать разнообразные обработчики событий, не только щелчки по кнопкам. «Кнопочный» интерфейс выглядит не современно.

3 Описание работы программы

В проекте были внесены изменения, направленные на добавление возможности сохранения и загрузки автопарка. Для этого в файле Form1.cs были созданы две новые кнопки: "Сохранить парк" и "Загрузить парк", которые позволяют пользователю сохранять текущий список автомобилей в файл и загружать его обратно.

```
private void buttonLoadCarPark_Click(object sender, EventArgs e)
{
    try
    {
        // Указываем путь к файлу
        string filePath = "CarPark.txt";

        if (!File.Exists(filePath))
        {
            MessageBox.Show("Файл с автопарком не найден.");
            return;
        }

        carPark.Cars.Clear(); // Очищаем текущий автопарк
        listBoxCars.Items.Clear(); // Очищаем список на форме

        using (var reader = new StreamReader(filePath))
        {
            string line;
            while ((line = reader.ReadLine()) != null)
            {
                // Парсим строку файла
                var data = line.Split(';');
                string name = data[0];
                int horsepower = int.Parse(data[1]);
                int fuelConsumption = int.Parse(data[2]);
                int fuelTankCapacity = int.Parse(data[3]);
                int fuel = int.Parse(data[4]);
                int mileage = int.Parse(data[5]);
                string engineCondition = data[6];
                var wheelConditions = data[7].Split(',');

                // Создаем новый объект Car
                var car = new Car(name, horsepower, fuelConsumption, fuelTankCapacity)
                {
                    Fuel = fuel,
                    Mileage = mileage
                };

                car.Engine.ChangeCondition(engineCondition);

                for (int i = 0; i < Car.WheelCount; i++)
                {
                    car.Wheels[i].ChangeCondition(wheelConditions[i]);
                }

                carPark.AddCar(car); // Добавляем машину в автопарк
                listBoxCars.Items.Add(name); // Добавляем название в список
            }
        }

        listBoxActions.Items.Add("Автопарк успешно загружен.");
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка при загрузке автопарка: {ex.Message}");
    }
}
```

Рисунок 1 – Метод загрузки из txt

Эти кнопки были добавлены в пользовательский интерфейс через файл Form1.Designer.cs. Кнопка "Сохранить парк" привязана к методу button1_Click, где реализована логика записи данных из объекта автопарка в файл. Кнопка "Загрузить парк" связана с методом buttonLoadCarPark_Click_1, в котором происходит чтение данных из файла и восстановление списка автомобилей в программе.

```

Сообщение 1
private void buttonSaveCarPark_Click(object sender, EventArgs e)
{
    try
    {
        // Указываем путь к файлу (можно сделать выбор пути через диалоговое окно)
        string filePath = "CarPark.txt";

        using (var writer = new StreamWriter(filePath))
        {
            // Сохраняем данные о каждом автомобиле в автопарке
            foreach (var car in carPark.Cars)
            {
                writer.WriteLine($"{car.Name};{car.Engine.HorsePower};{car.FuelConsumption};{car.FuelTankCapacity};{car.Fuel};{car.Mi}");
            }
        }

        listBoxActions.Items.Add("Автопарк успешно сохранен.");
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка при сохранении автопарка: {ex.Message}");
    }
}

```

Рисунок 2 – Метод выгрузки из txt

Для реализации функционала сохранения и загрузки в классе CarPark (файл CarPark.cs) добавлены два новых метода: SaveToFile(string filePath) и LoadFromFile(string filePath). Метод SaveToFile позволяет записать данные обо всех автомобилях автопарка в текстовый файл в удобном формате, а метод LoadFromFile обеспечивает обратный процесс — чтение данных из файла и добавление их в автопарк. Благодаря этим изменениям пользователи теперь могут сохранять текущий прогресс работы с автопарком и продолжать его позже, загружая сохраненные данные.

4 Исходный код программы

Ниже представлен исходный код программы.

Файл Form1.cs (приложение):

```
using System;

using System.IO;

using System.Linq;

using System.Windows.Forms;

using CarManagementApp;

namespace CarManagementApp
{
    public partial class Form1 : Form
    {
        // Объект автопарка, который будет хранить все автомобили

        private CarPark carPark = new CarPark();


        // Конструктор формы, инициализация компонентов

        public Form1()
        {
            InitializeComponent();
        }


        // Обработчик кнопки "Создать авто"
```

// При нажатии создает новый объект Car с параметрами из
текстовых полей, добавляет его в автопарк

```
private void buttonAddCar_Click(object sender, EventArgs e)
{
    try
    {
        string name = textBoxCarName.Text;

        int horsepower = int.Parse(textBoxHorsePower.Text);

        int fuelConsumption = int.Parse(textBoxFuelConsumption.Text);

        int fuelTankCapacity = int.Parse(textBoxFuelTankCapacity.Text);

        // Создаем объект типа Info вместо базового Car

        var car = new Info(name, horsepower, fuelConsumption,
fuelTankCapacity);

        carPark.AddCar(car);

        listBoxCars.Items.Add(name);

        listBoxActions.Items.Add($"Добавлен автомобиль: {name}");
    }

    catch (Exception ex)
    {
```

```
        MessageBox.Show($"Ошибка при добавлении автомобиля:  
{ex.Message}");  
    }  
}
```

```
// Обработчик кнопки "Удалить авто"  
  
// При нажатии удаляет выбранный автомобиль из автопарка  
private void buttonRemoveCar_Click(object sender, EventArgs e)  
{  
    try  
    {  
        // Получаем выбранную машину из списка  
        string selectedCar = listBoxCars.SelectedItem?.ToString();  
        if (selectedCar != null)  
        {  
            // Удаляем машину из автопарка  
            carPark.RemoveCar(selectedCar);  
  
            // Обновляем интерфейс: удаляем машину из списка и  
выводим действия  
            listBoxCars.Items.Remove(selectedCar);  
        }  
    }  
}
```

```

        listBoxActions.Items.Add($"Удален автомобиль:
{selectedCar}");

    }

    else

    {

        // Если машина не выбрана, выводим предупреждение

        MessageBox.Show("Выберите автомобиль для удаления.");

    }

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

    MessageBox.Show($"Ошибка при удалении автомобиля:
{ex.Message}");

}

}

```

// Обработчик кнопки "Запустить двигатель"

// При нажатии запускает двигатель выбранного автомобиля

```
private void buttonStartEngine_Click(object sender, EventArgs e)
```

```
{
```

```
    try
```

```
    {
```

```

// Получаем выбранную машину из списка

string selectedCar = listBoxCars.SelectedItem?.ToString();

if (selectedCar != null)
{
    // Получаем объект автомобиля из автопарка и запускаем
двигатель

    Car car = carPark.GetCar(selectedCar);

    car.StartEngine();

    // Обновляем интерфейс: выводим действия

    listBoxActions.Items.Add($"Двигатель автомобиля
{selectedCar} запущен.");

}

else
{
    // Если машина не выбрана, выводим предупреждение

    MessageBox.Show("Выберите автомобиль для запуска
двигателя.");

}

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

```

```

        MessageBox.Show($"Ошибка при запуске двигателя:
{ex.Message}");
    }
}

// Обработчик кнопки "Выключить двигатель"
// При нажатии выключает двигатель выбранного автомобиля
private void buttonStopEngine_Click(object sender, EventArgs e)
{
    try
    {
        // Получаем выбранную машину из списка
        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)
        {
            // Получаем объект автомобиля из автопарка и выключаем
двигатель

            Car car = carPark.GetCar(selectedCar);

            car.StopEngine();

            // Обновляем интерфейс: выводим действия
            listBoxActions.Items.Add($"Двигатель автомобиля
{selectedCar} остановлен.");
        }
    }
}

```

```

    }

    else

    {

        // Если машина не выбрана, выводим предупреждение
        MessageBox.Show("Выберите автомобиль для остановки
двигателя.");

    }

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при остановке двигателя:
{ex.Message}");

}

}

// Обработчик кнопки "Заправка"

// При нажатии заправляет выбранный автомобиль на указанное
количество топлива

private void buttonRefuel_Click(object sender, EventArgs e)

{

    try

    {

```

```

// Получаем выбранную машину из списка

string selectedCar = listBoxCars.SelectedItem?.ToString();

if (selectedCar != null)
{
    // Получаем количество топлива из текстового поля

    int fuelAmount = int.Parse(textBoxFuelAmount.Text);

    // Получаем объект автомобиля из автопарка и заправляем
его

    Car car = carPark.GetCar(selectedCar);

    car.Refuel(fuelAmount);

    // Обновляем интерфейс: выводим действия

    listBoxActions.Items.Add($"Автомобиль {selectedCar}
заправлен на {fuelAmount} литров.");

}

else

{
    // Если машина не выбрана, выводим предупреждение

    MessageBox.Show("Выберите автомобиль для заправки.");

}

}

catch (Exception ex)

```

```

    {

        // В случае ошибки выводим сообщение

        MessageBox.Show($"Ошибка при заправке автомобиля:
{ex.Message}");

    }

}

// Обработчик кнопки "Передвижение"

// При нажатии передвигает выбранный автомобиль на указанное
расстояние

private void buttonMove_Click(object sender, EventArgs e)
{
    try
    {

        // Получаем выбранную машину из списка

        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)
        {

            // Получаем расстояние из текстового поля

            int distance = int.Parse(textBoxDistance.Text);

            // Проверка, выбрал ли пользователь ускоренный режим
движения

```

```

bool isFast = checkBoxFastMode.Checked;

// Вызов перегруженного метода Move с параметром для
высокой скорости

if (isFast)
{
    // Перемещение с высокой скоростью

    Car car = carPark.GetCar(selectedCar);

    car.Move(distance, true); // Перегруженная версия метода
Move с параметром isFast

    listBoxActions.Items.Add($"Автомобиль {selectedCar}
проехал {distance} км с высокой скоростью.");
}
else
{
    // Стандартное перемещение

    Car car = carPark.GetCar(selectedCar);

    car.Move(distance); // Стандартная версия метода Move

    listBoxActions.Items.Add($"Автомобиль {selectedCar}
проехал {distance} км.");
}
}
else
{

```

```

        // Если машина не выбрана, выводим предупреждение
        MessageBox.Show("Выберите автомобиль для
передвижения.");
    }
}

catch (Exception ex)
{
    // В случае ошибки выводим сообщение
    MessageBox.Show($"Ошибка при движении автомобиля:
{ex.Message}");
}
}

// Обработчик кнопки "Заменить колесо"

// При нажатии заменяет сломанное колесо выбранного
автомобиля

private void buttonReplaceWheel_Click(object sender, EventArgs e)
{
    try
    {
        // Получаем выбранную машину из списка
        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)

```

```

    {

        // Получаем объект автомобиля из автопарка и заменяем
колесо

        Car car = carPark.GetCar(selectedCar);

        car.ReplaceBrokenWheel();


        // Обновляем интерфейс: выводим действия

        listBoxActions.Items.Add($"Колесо автомобиля {selectedCar}
заменено.");

    }

    else

    {

        // Если машина не выбрана, выводим предупреждение

        MessageBox.Show("Выберите автомобиль для замены
колеса.");

    }

}

catch (Exception ex)

{

    // В случае ошибки выводим сообщение

    MessageBox.Show($"Ошибка при замене колеса:
{ex.Message}");

}

}

```

```

// Обработчик кнопки "Показать информацию"

// При нажатии отображает информацию о выбранном автомобиле

private void buttonShowInfo_Click(object sender, EventArgs e)
{
    try
    {
        // Проверяем, был ли выбран автомобиль

        string selectedCar = listBoxCars.SelectedItem?.ToString();

        if (selectedCar != null)
        {
            Car car = carPark.GetCar(selectedCar);

            if (car == null)
            {
                MessageBox.Show("Ошибка: Автомобиль не найден.");

                return;
            }

            // Добавляем диагностику типа объекта

            if (car is Info infoCar)
            {
                // Если это объект типа Info, используем метод Info

```

```

        bool showDetails = checkBoxShowDetails.Checked;

        string info = infoCar.GetCarInfo(showDetails);

        MessageBox.Show(info);
    }

    else

    {

        // В противном случае используем метод базового класса

        bool showDetails = checkBoxShowDetails.Checked;

        string info = car.GetCarInfo(showDetails);

        MessageBox.Show(info);

    }

}

else

{

    MessageBox.Show("Выберите автомобиль для отображения
информации.");

}

}

catch (Exception ex)

{

    MessageBox.Show($"Ошибка при отображении информации:
{ex.Message}");

}

```

```
}
```

```
private void button1_Click(object sender, EventArgs e)
```

```
{
```

```
}
```

```
// Обработчик кнопки "Сохранить автопарк"
```

```
private void buttonSaveCarPark_Click(object sender, EventArgs e)
```

```
{
```

```
    try
```

```
    {
```

```
        // Указываем путь к файлу (можно сделать выбор пути через  
        диалоговое окно)
```

```
        string filePath = "CarPark.txt";
```

```
        using (var writer = new StreamWriter(filePath))
```

```
        {
```

```
            // Сохраняем данные о каждом автомобиле в автопарке
```

```
            foreach (var car in carPark.Cars)
```

```
            {
```

```
                writer.WriteLine($"{car.Name};{car.Engine.HorsePower};{car.FuelConsumption}
```

```
;{car.FuelTankCapacity};{car.Fuel};{car.Mileage};{car.Engine.Condition};{string  
.Join(",", car.Wheels.Select(w => w.Condition))}");
```

```
}
```

```
}
```

```
listBoxActions.Items.Add("Автопарк успешно сохранен.");
```

```
}
```

```
catch (Exception ex)
```

```
{
```

```
    MessageBox.Show($"Ошибка при сохранении автопарка:  
{ex.Message}");
```

```
}
```

```
}
```

```
// Обработчик кнопки "Загрузить автопарк"
```

```
private void buttonLoadCarPark_Click(object sender, EventArgs e)
```

```
{
```

```
    try
```

```
    {
```

```
        // Указываем путь к файлу
```

```
        string filePath = "CarPark.txt";
```

```
        if (!File.Exists(filePath))
```

```
{  
    MessageBox.Show("Файл с автопарком не найден.");  
    return;  
}
```

```
carPark.Cars.Clear(); // Очищаем текущий автопарк  
listBoxCars.Items.Clear(); // Очищаем список на форме
```

```
using (var reader = new StreamReader(filePath))  
{  
    string line;  
    while ((line = reader.ReadLine()) != null)  
    {  
        // Разбираем строку файла  
        var data = line.Split(';');  
        string name = data[0];  
        int horsepower = int.Parse(data[1]);  
        int fuelConsumption = int.Parse(data[2]);  
        int fuelTankCapacity = int.Parse(data[3]);  
        int fuel = int.Parse(data[4]);  
        int mileage = int.Parse(data[5]);  
        string engineCondition = data[6];  
        var wheelConditions = data[7].Split(',');
```

```

        // Создаем новый объект Car
        var car = new Car(name, horsePower, fuelConsumption,
fuelTankCapacity)
        {
            Fuel = fuel,
            Mileage = mileage
        };

        car.Engine.ChangeCondition(engineCondition);

        for (int i = 0; i < Car.WheelCount; i++)
        {
            car.Wheels[i].ChangeCondition(wheelConditions[i]);
        }

        carPark.AddCar(car); // Добавляем машину в автопарк
        listBoxCars.Items.Add(name); // Добавляем название в
список

    }

}

listBoxActions.Items.Add("Автопарк успешно загружен.");

```

```

    }

    catch (Exception ex)

    {
        MessageBox.Show($"Ошибка при загрузке автопарка:
{ex.Message}");
    }
}

private void buttonLoadCarPark_Click_1(object sender, EventArgs e)
{

}

}

}

```

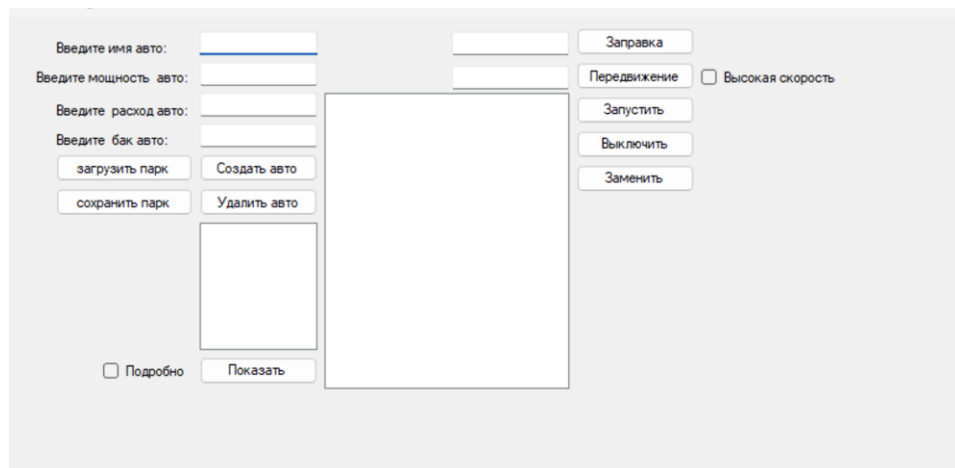


Рисунок 3 – Расположение графических элементов

5 Результат работы программы

При добавлении информации о книге, информация добавляется и в listBox, и в TextBox (Рисунок 4).

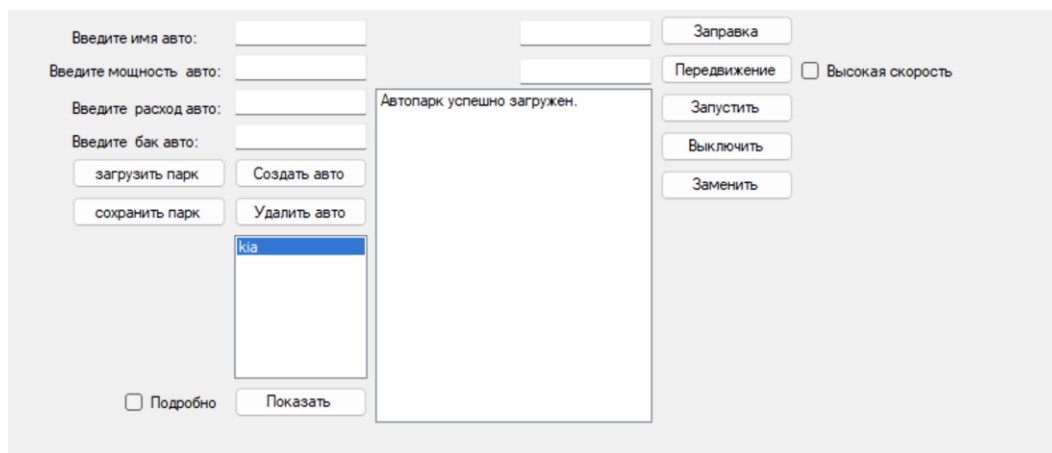


Рисунок 4 – Пример работы

При нажатии на кнопку «Сохранить» текстовая информация о всех книгах сохранится в отдельный файл (Рисунок 5).

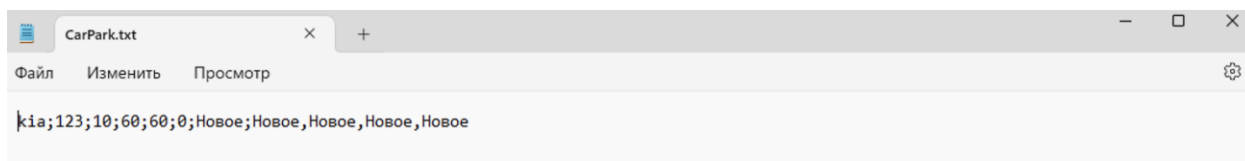


Рисунок 5 – Сохранение в файле

6 Вывод

В этой лабораторной работе мной были изучены способы чтения и записи текстовых файлов с помощью классов и их перегруженных версий из пространства имен Stream.IO.